# Lab 1: FFNN

*Group-13*

Amirali Eghdami Malati - s328630,
Hicham El kettani - s344121, Jingbin Hu - s336634

## Task 1 – Data Preprocessing

### *Remove missing values (NaN) and duplicate entries.*

The loaded tabular data initially had **31,507** entries and **16** features. We removed **2,111** duplicates which represent **6.7%** of the overall data. Afterwards there were found **20** rows (**0.06%**) with least one *NaN* value and **27** rows (**0.09%**) with at least one *inf/-inf* value (without taking into account how many are in common), which were all dropped as well. We decided not to impute the missing (*NaN* or *inf/-inf*) values because the number of affected rows is negligible. The resulted dataset has eventually **29,386** entries.

### *Categorical data transformation and Label distribution.*

All the attributes in the dataset are numerical. However the Label is of nominal categorical type which requires a numerical encoding, for that reason *Label encoding* was adopted; A simple and memory efficient encoding scheme, which assigns an integer to each category (class).

| **Label** | Benign | Brute Force | DoS Hulk | PortScan |
|---|---|---|---|---|
| **Encoding** | 0 | 1 | 2 | 3 |

Table 1: Label Encoding

The label distribution is highly imbalanced, with the *Benign* class dominating the dataset at roughly 65.5%, far larger than any of the attack categories. *PortScan* and *DoS Hulk* make up moderate portions (around 16.5% and 13.2%, respectively), while *Brute Force* represents a very small minority at only 4.9%.

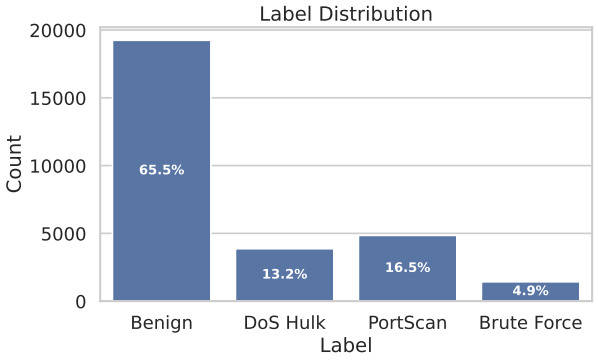



Figure 1: Label Distribution

### *Split the dataset to extract a training, validation and test sets.*

We split the dataset into *training set* (60%), *validation set* (20%), and *test set* (20%) using a stratified sampling approach to generate balanced and representative partitions without replacement, given the high imbalance of the original dataset. Fixing the **random_state** parameter (e.g, 42) the thing

that guarantees reproducibility, along with maintaining the class proportions of the initial dataset, are considered essential for reliable model training and unbiased evaluation.

### *Focus on the training and validation partitions. Check for the presence of outliers and decide on the correct normalization.*

The boxplots of the features in training and validation sets per class demonstrate the existence of big amount of outliers. These values are not removed as they are not negligible compared to the size of the dataset, as well as they are found to be realistic with respect to the nature of the flows. For these reasons and more, we adopted **z-score Normalization** (Standardization) as a numerical data transformation scheme of the features since it is more robust in the presence of outliers compared to min-max Scaling. As a result, the features are standardized to zero mean ($\mu$=0) and unit variance ($\sigma$=1).

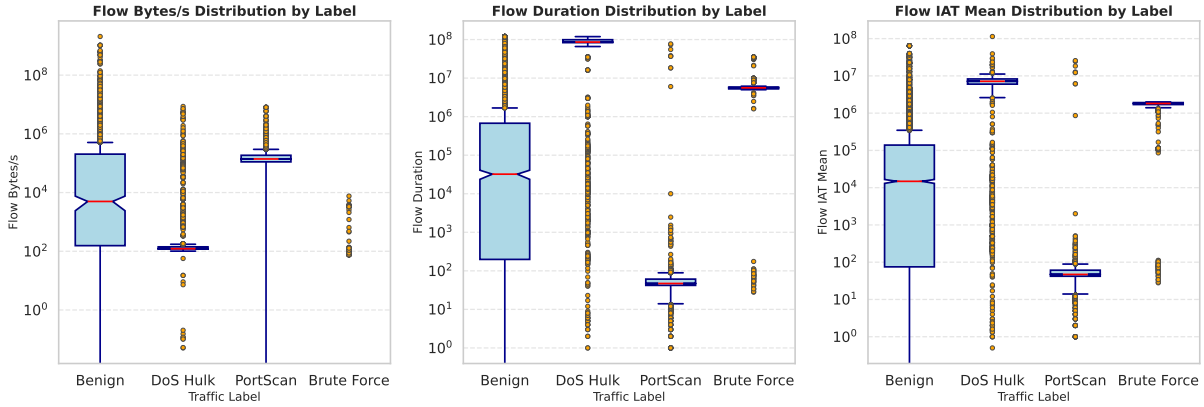
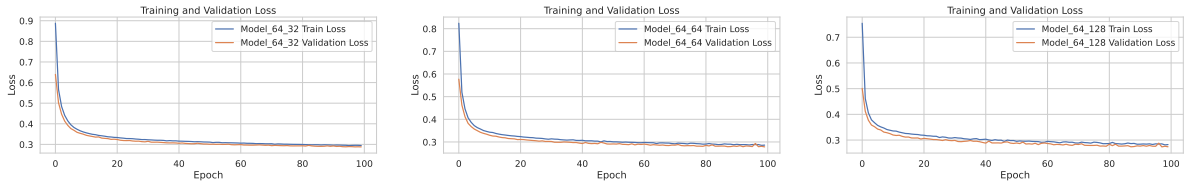



Figure 2: Boxplots of some features in combined validation and training sets

## Task 2 – Shallow Neural Network

### *Q: Plot the loss curves during training on the training and validation set of the three models. What is their evolution? Do they converge?*

The preprocessed data in Task1 were used to train three models based on different single-layer neural networks. The three models are characterized with the same specified hyper-parameters with the exception of the number of the hidden-layer's neurons. During training, the plotted loss curves of the three models on the training and validation sets show smooth evolution, with little perturbations in the curves of the model with 128 neurons. According to these curves, the bigger the number of neurons the faster the convergence toward better loss values. However all the three models seem to converge to a loss plateau around 0.28.




(a) Model1: 32 Neurons (b) Model2: 64 Neurons (c) Model3: 128 Neurons

Figure 3: Training and Validation losses through epochs of single-layer Shallow FFNN.

### *Q: How do you select the best model across epochs?*

To select the best model, we should monitor the validation losses and then select the model with the lowest converging validation loss, making sure that there is no underfitting or overfitting. The early stop mechanism may also be employed to capture the best checkpoint of the training process.

### *Q: How is the performance of the validation reports across the different classes? Is the performance good or poor? Why?*

The classification reports of the three models' validation sets all succeeded to achieve a high accuracy about **88%**, which might be a misleading metric due to the high class imbalance of the dataset. The **F1** score would reveal the performance more accurately; All three models completely failed to identify the minority class **(i.e. Brute Force)**. On the other hand, the other classes showed strong performance in all the models with high F1-score values, given their higher support. This behavior is primarily because linear models cannot learn complex decision boundaries; the model reduces to a single affine transformation, which lacks expressive power in imbalanced multi-class problems.

### *Q: Now, focus on the best model you chose. Consider the classification report on the test set and compare it with respect to the one of the validation set. Is the performance similar? I.e., does the model generalize?*

Selecting the best model was based on the validation set's F1-score values. The classification reports of both validation and test sets are very similar; they have got the same accuracy and very close F1-score values. The thing that confirms good model generalization.

| **Class** | **Validation Set** | | | **Test Set** | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1-score | Precision | Recall | F1-score |
| Benign | 0.90 | 0.94 | 0.92 | 0.90 | 0.94 | 0.92 |
| Brute Force | 0.36 | 0.03 | 0.06 | 0.48 | 0.05 | 0.09 |
| DoS Hulk | 0.99 | 0.88 | 0.93 | 0.99 | 0.87 | 0.93 |
| PortScan | 0.77 | 0.92 | 0.84 | 0.77 | 0.92 | 0.84 |
| **Accuracy** | | 0.88 | | | 0.88 | |
| **Macro Avg** | 0.76 | 0.69 | 0.69 | 0.79 | 0.70 | 0.70 |
| **Weighted Avg** | 0.86 | 0.88 | 0.86 | 0.87 | 0.88 | 0.87 |

Table 2: Classification performance summary for Model_64_128 on validation and test sets.

### *Q: Focus and report the classification report of the validation set (ReLu as activation function). Does the model perform better in a specific class?*

After introducing non-linearity to the model by using *ReLu* as an activation function, the performance (accuracy and F1-scores) is enhanced compared to the previous model. This can be explained by the fact that non-linearity favors learning more complex patterns of the dataset especially of the rarest class. The classes now have higher and more comparable F1-scores, although the *Brute Force* class still has a slightly lower performance compared to the remaining classes due to its low support.

| **Class** | **Precision** | **Recall** | **F1-score** | **Support** |
|---|---|---|---|---|
| Benign | 0.97 | 0.97 | 0.97 | 3848 |
| Brute Force | 0.78 | 0.94 | 0.85 | 286 |
| DoS Hulk | 0.98 | 0.94 | 0.96 | 773 |
| PortScan | 0.94 | 0.92 | 0.93 | 970 |
| **Accuracy** | | 0.95 | | 5877 |
| **Macro Avg** | 0.92 | 0.94 | 0.93 | 5877 |
| **Weighted Avg** | 0.96 | 0.95 | 0.95 | 5877 |

Table 3: Classification performance summary for Model_64_128 using ReLu on the validation set.

### *Q: Would it be correct to compare the results on the test set?*

It is not correct to compare the results of the test set during hyper-parameters tuning, especially after changing the activation function. The comparison should be carried out with the validation set, so there would be no information leakage from the test set into the model selection process.

## Task 3 – The impact of Specific Features

### *Q: Is this a reasonable assumption?*

It is unreasonable to assume that all *Brute Force* attacks originate from a unique port, since in real world scenarios, these kind of attacks can target different ports. In our dataset these attacks are only associated with port 80, the thing which will introduce a strong data collection bias, and would lead the model to associate this port directly to *Brute Force* attacks rather than learning its real patterns.

### *Q: Considering the validation classification report, does the performance change? How does it change? Why?*

After replacing port 80 with port 8080 for the *Brute Force* attacks in the *Test set*, the classification report shows a sharp drop in the performance in classifying *Brute Force* attacks; The F1-score of this class dropped from **0.86** to **0.08**, and the accuracy decreased to **91%**. The drop in performance demonstrates that the model has learned a spurious shortcut, a known phenomenon called *shortcut learning*, which happens when the model learns easy but misleading patterns in the data, instead of the true features needed for proper generalization.

### *Q: How many PortScan do you now have after preprocessing (e.g., removing duplicates)? How many did you have before?*

After removing the feature *port* from the original dataset and repeating all pre-processing steps, **6,917 (23.54%)** duplicates overall have been removed from the dataset, among them **4,564** rows belong to *PortScan* class, resulting in only **22,469** rows overall, including **285** of the *PortScan* class.

### *Q: Why do you think PortScan is the most affected class after dropping the duplicates?*

As stated before, Among **6,917** duplicates , **4,564** of them belong to only the *PortScan* class. This latter had the port as its only distinguishing feature; All the flows of this class has the same characteristics but directed towards different ports. Removing the port feature collapsed the rows into duplicates.

### *Q: Are the classes now balanced?*

After removal of duplicates, the classes become more imbalanced, and *PortScan* is the rarest one with percentage of only **1.27%**.

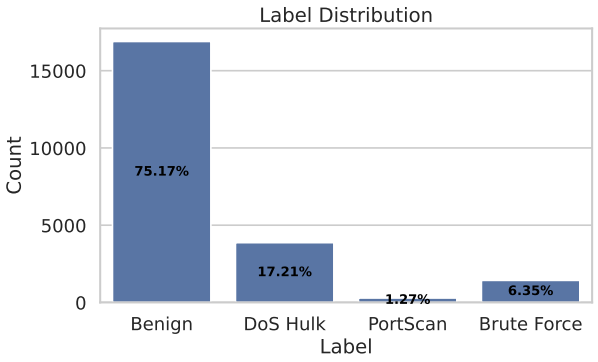



Figure 4: Label Distribution after removing port feature and pre-processing.

# Task 4 – The impact of the Loss Function

### *Q: How does the performance change? Can you still classify the rarest class?*

After removing the *Destination Port* feature and re-processing the dataset, we trained the best model (128 Neurons) from the previous step. The performance of the model in classifying *PortScan* class is dropped; The F1-score of this class reached a value of **0.58**. Now it become harder to classify the rarest class due to its low support. Although the general accuracy of the model remains high at **95%**.

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Benign | 0.97 | 0.97 | 0.97 | 3378 |
| Brute Force | 0.82 | 0.95 | 0.88 | 285 |
| DoS Hulk | 0.99 | 0.90 | 0.94 | 774 |
| PortScan | 0.55 | 0.61 | 0.58 | 57 |
| **Accuracy** | | 0.95 | | 4494 |
| **Macro Avg** | 0.83 | 0.86 | 0.84 | 4494 |
| **Weighted Avg** | 0.96 | 0.95 | 0.96 | 4494 |

Table 4: Classification performance summary for Model_64_128 on the validation set.

### *Q: Which partition do you use to estimate the class weights?*

We must use the **Training Set**; Class weights are training parameters designed to help the model learn from the specific distribution it is seeing during the optimization process. Using the Validation or Test set to calculate these weights would constitute data leakage, as information about the distribution of unseen data would influence the model's training. To ensure a fair evaluation, all statistics used for preprocessing (like scaling) or configuration (like class weights) must be derived solely from the training data.

### *Q: How does the performance change per class and overall? In particular, how does the accuracy change? How does the f1 score change?*

Using *weighted loss* improved significantly recall for *PortScan* class, from **0.61** to **0.93**, which allows the model to detect more of these rare samples in a robust way. Nonetheless, its precision is dropped from **0.55** to **0.28**, which means that more false positives were found. The accuracy instead remains high with a slight drop of only 2%. While the F1-score of the rarest class decreased to **0.43**, the other classes maintained the same values with a very slight drop of at most **0.01**.

# Task 5 – Deep Neural Network

## 5.1 Design the first Deep Learning

We evaluated six different feedforward neural network architectures by varying depth, width, and layer configurations; Each model name (architecture) is described as the following:
Model_{batch size}_{#Layers}_{inc | dec}, where *inc* (*dec*) means that the number of neurons is increased (decreased) per layer starting from 2 (32) neurons.

#### *Q: Plot and analyze the losses. Do the models converge?*

The loss curves of the models show different trends. *Model_64_3_dec* and *Model_64_4_dec* have the fastest loss drop with smooth converging training and validation losses. All the other models didn't converge completely and their curves still retain higher loss values, suggesting underfitting or excessive compression leading to degraded learning capacity.

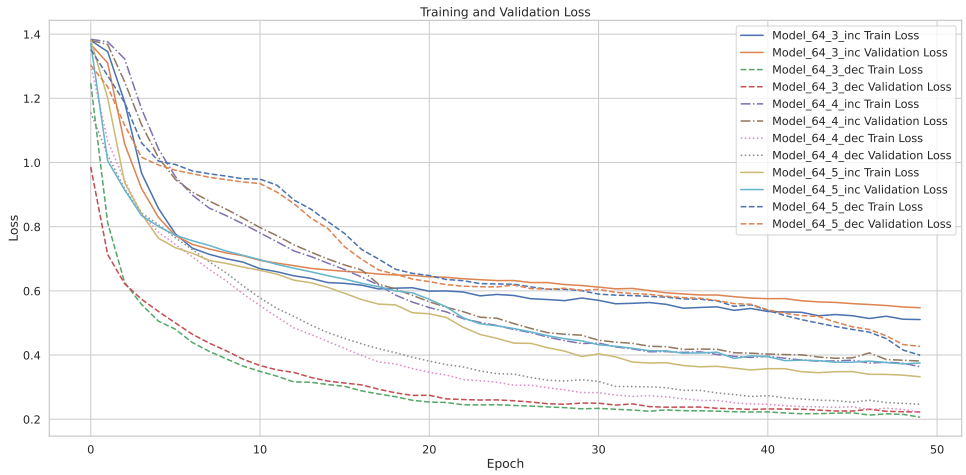



Figure 5: Training and Validation losses through epochs of Deep FFNNs.

***Q: Calculate the performance in the validation set and identify the best-performing architecture. How do you select one?***

We selected **Model_64_4_dec** as the best-performing architecture based on the following criteria:

- **Overall Accuracy & F1-Score:** It achieved the highest overall accuracy (**94%**) and highest Macro (Weighted) F1-score with value **0.81** (**0.94**).
- **Handling Class Imbalance:** The dataset is highly imbalanced, with *PortScan* class having significantly fewer samples. *Model_64_4_dec* demonstrated superior performance on this minority class (F1-score of **0.52**) compared to other models, indicating better generalization capabilities.
- **Loss Stability:** As observed in the validation loss plot, *Model_64_4_dec* achieved a low and stable loss, indicating effective learning without overfitting.

| Model | Accuracy | Macro F1-Score | Weighted F1-Score | PortScan F1-Score |
|---|---|---|---|---|
| Model_64_3_inc | 82% | 0.65 | 0.85 | 0.22 |
| Model_64_3_dec | 91% | 0.77 | 0.92 | 0.40 |
| Model_64_4_inc | 81% | 0.64 | 0.85 | 0.20 |
| **Model_64_4_dec** | **94%** | **0.81** | **0.94** | **0.52** |
| Model_64_5_inc | 89% | 0.73 | 0.91 | 0.29 |
| Model_64_5_dec | 85% | 0.71 | 0.90 | 0.16 |

Table 5: Performance summary of different neural network architectures on the Validation Set.

***Evaluate and report the performance of the best model in the test set.***

To verify the generalization capability of the selected best model (Model_64_4_dec), we evaluated it on the independent test set. The model achieved a test accuracy of **93%**, which is nearly identical to the validation accuracy (94%), and has the same average F1-scores as of those of validation set, confirming excellent generalization with no overfitting. Regarding class-specific performance, despite *PortScan* being a minority (only 57 samples), the model maintained a consistent F1-score of **0.51**. The selected architecture is therefore validated as effective and stable for this task.

| Class | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| Benign | 0.97 | 0.94 | 0.96 | 3378 |
| Brute Force | 0.71 | 0.95 | 0.82 | 286 |
| DoS Hulk | 0.97 | 0.90 | 0.93 | 773 |
| PortScan | 0.36 | 0.84 | 0.51 | 57 |
| **Accuracy** | | 0.93 | | 4494 |
| **Macro Avg** | 0.75 | 0.91 | 0.80 | 4494 |
| **Weighted Avg** | 0.95 | 0.93 | 0.94 | 4494 |

Table 6: Classification performance summary for Model_64_4_dec on the Test Set.

## 5.2 The impact of Batch Size

Using the best deep architecture (Model_64_4_dec), we trained with batch sizes 4, 64, 256, 1024.

***Q: Does performance change? And why? Report the validation results.***

The loss curves show that batch size 4 achieved faster convergence compared to larger batches, which converged slower and plateaued at higher losses. The high variance in gradient estimation introduced by small batches helps the optimizer to escape sharp, suboptimal local minima and converge to flatter, more generalizable solutions. Conversely, the highly stable gradients from large batches often lead to premature convergence to less optimal states.

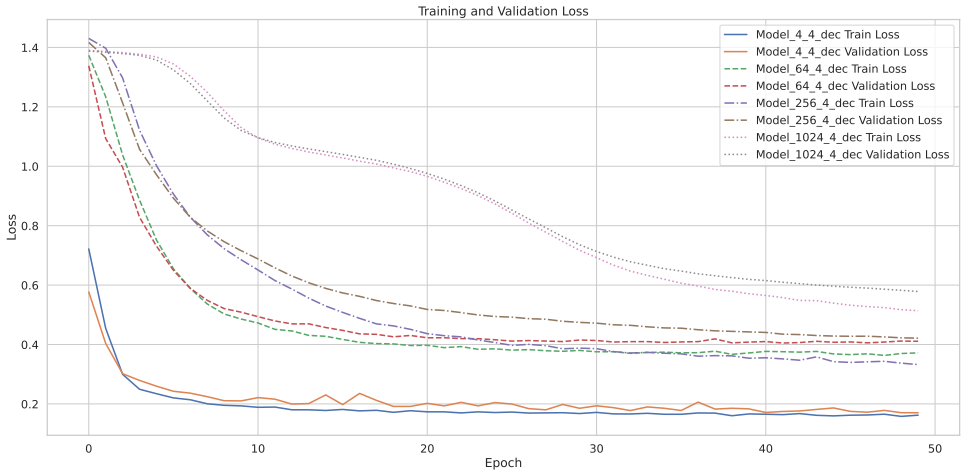



Figure 6: Training and Validation losses of Model_{..}_4_dec .

Training the model with batch size of 4 introduced stochastic noise into gradient updates, the thing that helped to obtain the best performance with accuracy equal to 95% and higher average F1-Scores. This configuration also was the most suited to classify the rarest class (*PortScan*) with F1-Score equal to **0.55**.

| Batch Size | Accuracy | Macro F1-Score | PortScan F1-Score | Training Time (s) |
|---|---|---|---|---|
| 4 | 95% | 0.83 | 0.55 | 393.5 |
| 64 | 93% | 0.79 | 0.50 | 29 |
| 256 | 91% | 0.76 | 0.35 | 11.4 |
| 1024 | 81% | 0.68 | 0.14 | 7.7 |

Table 7: Performance summary of different Batch Sizes on the Validation Set.

***Q: How long does it take to train the models depending on the batch size? And why?***

Training time decreased consistently as the batch size increased. Larger batches allow more parallel computation and fewer weight updates per epoch, leading to faster overall training despite potential trade-offs in generalization.

## 5.3 The impact of the Optimizer

We trained the best model using a batch size of 64 to speed up the training with the following optimizers: SGD, SGD with momentum (0.1, 0.5, 0.9), and AdamW.

***Q: Is there a difference in the trend of the loss functions?***

**Yes, the trends differ significantly**. As shown in Figure.7, *AdamW* exhibits the fastest convergence, dropping immediately to the lowest loss ($\approx 0.2$). *SGD with high momentum (0.9)* converges effectively but at a much slower rate. In contrast, *Vanilla SGD* and low-momentum variants stagnate at high loss values, failing to overcome the initial plateau.

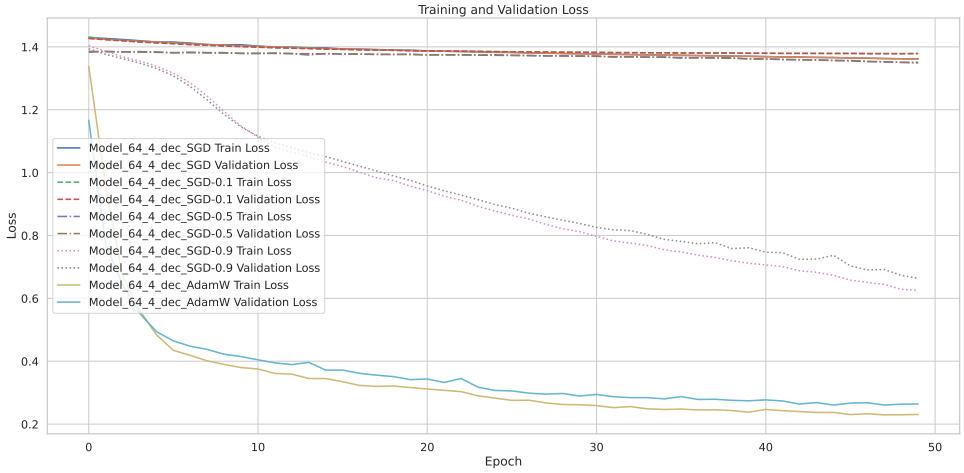



Figure 7: Training and Validation losses of Model_64_4_dec with different optimizers.

***Q: How long does it take to train the models with the different optimizers? And why?***

Increasing the momentum in *SGD* leads generally to both better performance and slightly longer training time. This is because momentum smooths gradient updates, improving convergence but requiring additional computation. *AdamW* further improves performance by combining momentum with gradient normalization, adapting learning rates for each parameter. This results in the best validation loss and accuracy, though it is the slowest due to the added complexity of its updates.

***Evaluate the effects of the different learning rates and epochs. Report the test results for the best model.***

We trained the model Model_64_4_dec_AdamW with different learning rates: {0.0005, 0.0001, 0.001, 0.01, 0.1}. Higher learning rates either remain trapped in the first plateau failing to decrease the loss (**lr=0.1**), or manage to achieve a reasonable low loss but with big oscillations (**lr=0.01**). On the other hand, with small learning rates (**lr=0.0005, 0.0001**), the loss curves are very smooth but converge slowly, possibly struggling to escape local minima. Training the model with learning rate of **0.001** achieved a tradeoff, and ultimately resulted in the fastest converging curves and obtained the lowest loss values. After increasing the number of epochs, the model with highest learning rate is still stuck in a local minimum, and others are still didn't reach a plateau (**lr=0.0001**), while others start degenerating, resulting in higher losses (**lr=0.01**). With more epochs, even the well performing models (**lr=0.001, 0.0005**) start degenerating as well showcasing some overfitting.
The configuration with **LR=0.001** and **Epochs=100** was selected as the optimal model. As shown in the plot, **LR=0.001** demonstrates the most stable and efficient *"L-shaped"* convergence, avoiding the

instability of LR=0.01 and the slow learning of LR=0.0001. While the model stabilized around epoch 50, extending training to 100 epochs yielded the highest validation F1-score, particularly improving the recognition of minority classes without overfitting.

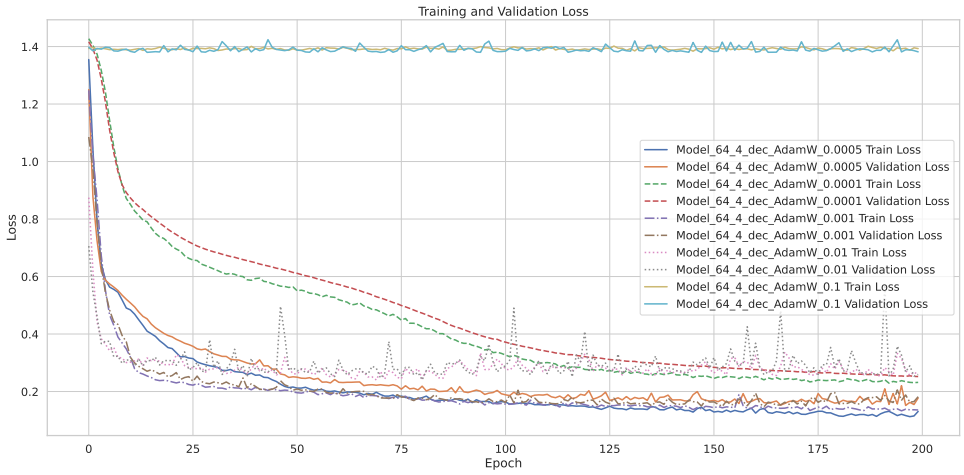



Figure 8: Training and Validation losses of Model_64_4_dec_AdamW with different learning rates.

| Learning Rate | 50 Epochs | | 100 Epochs | | 200 Epochs | |
|---|---|---|---|---|---|---|
| | Val Acc | Macro F1 | Val Acc | Macro F1 | Val Acc | Macro F1 |
| 0.0001 | 68% | 0.56 | 89% | 0.75 | 93% | 0.80 |
| 0.0005 | 92% | 0.77 | 92% | 0.77 | 93% | 0.79 |
| 0.001 | 90% | 0.77 | 95% | 0.83 | 93% | 0.79 |
| 0.01 | 87% | 0.72 | 87% | 0.70 | 88% | 0.72 |
| 0.1 | 17% | 0.07 | 75% | 0.21 | 17% | 0.07 |

Table 8: Validation loss and test accuracy across learning rates and training durations.

# Task 6 – Overfitting and Regularization

We trained the specified 6-layer FFNN, then applied dropout, batch normalization, and weight decay to observe their impact on overfitting and model performance.

### *Q: What do the losses look like? Is the model overfitting?*

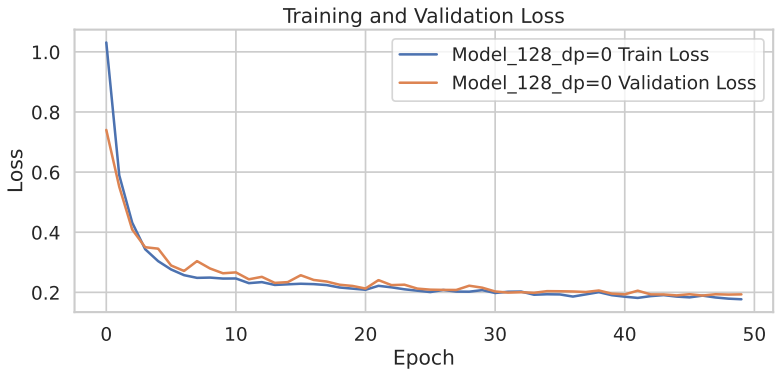



Figure 9: Training and Validation losses of Model_128 (No Normalization/Regularization).

Both the training and validation loss curves have an *"L-shaped"* form, converging to a plateau of value less then 0.2. This pattern suggests that the model is learning effectively and is not overfitting. It is also confirmed from the almost identical performance values of the validation and test sets.

### *Q: What impact do the different normalization techniques have on validation and testing performance?*

The different normalization and regularization techniques had varied impacts on performance. The evaluation reveals that the baseline model consistently provides the strongest performance, indicating that the network does not suffer from severe overfitting. Introducing mild dropout (0.2) produces only small changes in validation accuracy and F1-scores, but higher dropout levels (0.4, 0.5) lead to a noticeable degradation across all metrics, reflecting clear underfitting due to excessive regularization. Weight decay shows a similar pattern: a small penalty (0.0001) has minimal or slightly positive effects, whereas a larger decay (0.01) generally harms performance. Batch Normalization, however, systematically worsens results by raising validation loss and lowering both accuracy and F1-scores, especially for the minority *PortScan* class. This suggests that BN is not well suited for this fully connected architecture and tabular data, where batch statistics are unstable. Overall, the results indicate that strong regularization reduces model capacity unnecessarily, while the baseline or lightly regularized models achieve the best balance between stability and generalization.

| **Setup** | **Last Val Loss** | **Val Accuracy** | **Macro F1** | **PortScan F1** |
|---|---|---|---|---|
| Baseline | 0.1933 | 94% | 0.82 | 0.48 |
| Model_128_dp=0.2_wd=0 | 0.2036 | 93% | 0.78 | 0.37 |
| Model_128_dp=0.2_wd=0.01 | 0.2078 | 90% | 0.78 | 0.50 |
| Model_128_dp=0.2_wd=0.0001 | 0.2061 | 93% | 0.80 | 0.48 |
| Model_128_dp=0.2_bn_wd=0 | 0.2221 | 91% | 0.75 | 0.26 |
| Model_128_dp=0.2_bn_wd=0.01 | 0.2041 | 91% | 0.76 | 0.29 |
| Model_128_dp=0.2_bn_wd=0.0001 | 0.1984 | 92% | 0.77 | 0.33 |
| Model_128_dp=0.4_wd=0 | 0.2256 | 90% | 0.78 | 0.51 |
| Model_128_dp=0.4_wd=0.01 | 0.2276 | 90% | 0.78 | 0.50 |
| Model_128_dp=0.4_wd=0.0001 | 0.2318 | 89% | 0.76 | 0.43 |
| Model_128_dp=0.4_bn_wd=0 | 0.2251 | 90% | 0.75 | 0.34 |
| Model_128_dp=0.4_bn_wd=0.01 | 0.2187 | 93% | 0.79 | 0.43 |
| Model_128_dp=0.4_bn_wd=0.0001 | 0.2247 | 90% | 0.76 | 0.42 |
| Model_128_dp=0.5_wd=0 | 0.2351 | 89% | 0.73 | 0.24 |
| Model_128_dp=0.5_wd=0.01 | 0.2382 | 90% | 0.78 | 0.49 |
| Model_128_dp=0.5_wd=0.0001 | 0.2371 | 89% | 0.78 | 0.51 |
| Model_128_dp=0.5_bn_wd=0 | 0.2379 | 87% | 0.73 | 0.35 |
| Model_128_dp=0.5_bn_wd=0.01 | 0.2477 | 87% | 0.73 | 0.34 |
| Model_128_dp=0.5_bn_wd=0.0001 | 0.2410 | 89% | 0.76 | 0.46 |

Table 9: Performance comparison of different regularization techniques.